

TrueTone - AI Skin Analysis System

A deep learning-powered skin analysis system that predicts **Skin Type**, **Skin Tone**, and **Skin Disease** from a single image using three EfficientNet models running in parallel via a Django REST API.

Table of Contents

- Overview
 - Project Structure
 - Models Used
 - How the Pipeline Works
 - API Endpoints
 - Frontend Interface
 - Edge Cases Handled
 - How to Run
 - Testing
 - Configuration
 - File-by-File Breakdown
 - API Response Format
 - Architecture Diagram
 - Dependencies
-

Overview

TrueTone is a Final Year Project (FYP) that analyzes skin images using 3 separate deep learning models. The user uploads an image (through the web interface or API), and the system returns:

1. **Skin Type** - combination, dry, normal, or oily
2. **Skin Tone** - dark, fair, or medium
3. **Skin Disease** - common acne, cystic acne, eczema, psoriasis, rosacea, tinea, or none

All 3 models run **in parallel** using Python's **ThreadPoolExecutor**, meaning inference takes only as long as the slowest model (~200-330ms) rather than the sum of all three.

The system is built as a **Django web application** with REST API endpoints that accept image uploads and return JSON predictions.

Project Structure

FYP 26/

```
|
|-- manage.py                # Django management script (start server, etc.)
|-- pipeline_engine.py       # Core ML engine (model loading, inference, validation)
|-- test_api.py              # API test script (tests all endpoints)
|-- inspect_checkpoint.py    # Utility to inspect .pth/.pt checkpoint files
|-- requirements.txt         # Python dependencies
|-- README.md                # This file
|
|-- truetone/                # Django project settings
|   |-- __init__.py
|   |-- settings.py          # Django configuration (no database, CORS, uploads)
|   |-- urls.py              # Root URL routing
|   |-- wsgi.py              # WSGI entry point (for deployment)
|   |-- asgi.py              # ASGI entry point (for async deployment)
|
|-- api/                     # Django app for API endpoints
|   |-- __init__.py
|   |-- apps.py              # App configuration
|   |-- views.py             # API views (analyze, health, models, frontend)
|   |-- urls.py              # API URL routing
|   |-- middleware.py        # CORS middleware for cross-origin requests
|
|-- templates/
|   |-- index.html           # Frontend upload page (dark glassmorphism UI)
|
|-- models/                  # Trained model checkpoints (NOT Django models)
|   |-- skin_type/
|   |   |-- best_model.pth   # EfficientNet-B0 (torchvision) - 17.6 MB
|   |-- skin_tone/
|   |   |-- best_model.pt    # EfficientNet-B2 (timm) - 102.6 MB
|   |-- skin_disease/
|   |   |-- best_model.pth   # EfficientNet-B3 (torchvision) - 43.3 MB
|
|-- test_images/             # Test dataset images
|   |-- 000006.jpg
|   |-- 000014.jpg
|   |-- 000045.jpg
|   |-- 001021.jpg
|   |-- test_face_1.jpg
|
|-- static/                  # Static files directory (for CSS/JS if needed)
```

Models Used

Model	Architecture	Backend	Classes	Checkpoint	Size
Skin Type	EfficientNet-B0	torchvision	4: combination, dry, normal, oily	<code>best_model.pth</code>	17.6 MB
Skin Tone	EfficientNet-B2	timm	3: dark, fair, medium	<code>best_model.pt</code>	102.6 MB
Skin Dis-ease	EfficientNet-B3	torchvision	7: common_acne, cystic_acne, eczema, psoriasis, rosacea, tinea, none	<code>best_model.pth</code>	43.3 MB

Important Notes: - Skin Tone model uses `.pt` extension (NOT `.pth`) - Skin Tone model uses the `timm` library (NOT `torchvision`) - All models were trained with ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) - All models expect 224x224 input images

How the Pipeline Works

Complete Flow (Step by Step)

User uploads image (browser or API)

```

|
v
[1] DJANGO VIEW (api/views.py)
    - Receives the HTTP POST request
    - Extracts the uploaded file from request.FILES['image']
    - Performs basic server-side checks (file exists, not empty, size < 10MB)
    - Reads raw bytes from the file
    - Calls pipeline_engine.get_pipeline().analyze(bytes, filename)
|
v
[2] PIPELINE ENGINE - VALIDATION (pipeline_engine.py)
    - Checks file extension (.jpg, .jpeg, .png, .webp, .bmp only)

```

- Checks file size (max 10 MB)
- Opens image with PIL and verifies it's not corrupt
- Converts to RGB (handles grayscale, RGBA, etc.)
- Checks minimum resolution (64x64 pixels)
- Returns error with specific error_code if any check fails

|

v

[3] PIPELINE ENGINE - PREPROCESSING

- Resizes image to 256x256
- Center crops to 224x224
- Converts to PyTorch tensor
- Normalizes with ImageNet mean/std values
- Adds batch dimension [1, 3, 224, 224]
- This is done ONCE and shared by all 3 models

|

v

[4] PARALLEL INFERENCE (ThreadPoolExecutor with 3 workers)

- Submits 3 inference jobs simultaneously:
 - Thread 1: Skin Type model
 - Thread 2: Skin Tone model
 - Thread 3: Skin Disease model
- Each thread runs `_infer_one()`:
 1. Moves tensor to device (CPU/GPU)
 2. Forward pass through model (no gradient computation)
 3. Applies softmax to get probabilities
 4. Finds top prediction and confidence score
 5. Checks if confidence is below threshold
- Wall-clock time = time of SLOWEST model (not sum of all 3)

|

v

[5] RESULT AGGREGATION

- Combines results from all 3 models
- Generates warnings for low-confidence predictions
- Sets `disease_detected` flag (true only if disease != "none" AND confidence >= 40%)
- Records latency breakdown (preprocess, each model, parallel wall, total)
- Assigns unique request_id for tracking

|

v

[6] DJANGO VIEW - RESPONSE

- Returns JsonResponse with appropriate HTTP status code
- 200: Success
- 400: Invalid image / validation error
- 207: Partial error (some models failed, some succeeded)
- 500: Internal server error
- 503: Models not loaded

Why Parallel is Faster

Serial: [Type: 100ms] --> [Tone: 150ms] --> [Disease: 200ms] = 450ms total

Parallel: [Type: 100ms]
 [Tone: 150ms] --> all run at same time --> ~200ms total
 [Disease: 200ms]

The parallel pipeline typically achieves 1.5x-2x speedup over serial execution.

Model Auto-Detection

The pipeline automatically detects the correct model architecture from each checkpoint file:

1. **Backend detection:** Checks if `state_dict` keys start with `backbone.` (timm) or not (torchvision)
2. **Architecture detection:** Reads `in_features` of the first classifier Linear layer:
 - 1280 = EfficientNet-B0
 - 1408 = EfficientNet-B2
 - 1536 = EfficientNet-B3
 - 1792 = EfficientNet-B4
3. **Head detection:** Reads all `classifier.*` keys to determine:
 - Number of Linear layers (simple head vs. hidden layers)
 - Hidden dimension (if multi-layer head exists)
 - Output classes (from last Linear layer's shape)
4. **Checkpoint format:** Handles multiple save formats:
 - `model_state_dict` key (most common)
 - `state_dict` key (torchvision style)
 - `model_state` key (alternative)
 - Plain `state_dict` (bare weights)
 - Full serialized model (legacy)

This means you do NOT need to manually specify architectures. The engine figures it out automatically from the checkpoint.

Singleton Pattern

Models are loaded **once** and reused for all requests:

```
# First call: loads all 3 models (~5-15 seconds)
pipeline = get_pipeline()

# All subsequent calls: instant (returns same instance)
pipeline = get_pipeline()
```

This is implemented using a thread-safe double-checked locking pattern to prevent race conditions when multiple Django requests arrive simultaneously during

startup.

API Endpoints

1. POST /api/analyze - Analyze Skin Image

The main endpoint. Accepts an image file and returns predictions from all 3 models.

Request:

POST /api/analyze

Content-Type: multipart/form-data

Body: image=<file>

Success Response (200):

```
{
  "status": "success",
  "request_id": "a1b2c3d4",
  "image_info": {
    "filename": "face.jpg",
    "size_bytes": 45321,
    "width": 512,
    "height": 512
  },
  "predictions": {
    "skin_type": {
      "label": "oily",
      "confidence": 0.8734,
      "distribution": {
        "combination": 0.0512,
        "dry": 0.0234,
        "normal": 0.0520,
        "oily": 0.8734
      }
    },
    "skin_tone": {
      "label": "medium",
      "confidence": 0.6721,
      "distribution": {
        "dark": 0.1205,
        "fair": 0.2074,
        "medium": 0.6721
      }
    },
    "skin_disease": {
```

```

        "label": "none",
        "confidence": 0.9102,
        "disease_detected": false,
        "distribution": {
            "common_acne": 0.0108,
            "cystic_acne": 0.0201,
            "eczema": 0.0089,
            "psoriasis": 0.0156,
            "rosacea": 0.0210,
            "tinea": 0.0134,
            "none": 0.9102
        }
    },
    "skin_profile": {
        "type": "oily",
        "tone": "medium",
        "disease": "none",
        "disease_detected": false
    },
    "warnings": [],
    "latency_ms": {
        "preprocess_ms": 8.5,
        "skin_type_ms": 120.3,
        "skin_tone_ms": 145.2,
        "skin_disease_ms": 180.1,
        "parallel_wall_ms": 185.0,
        "total_ms": 193.5
    }
}

```

Error Response (400):

```

{
    "status": "error",
    "error_code": "UNSUPPORTED_FORMAT",
    "message": "Unsupported format '.gif'. Allowed: .bmp, .jpg, .jpeg, .png, .webp"
}

```

2. GET /api/health - Health Check

Returns server status, device info, model load status, and uptime.

Response (200):

```

{
    "status": "healthy",
    "device": "cpu",

```

```

    "models_loaded": {
        "skin_type": true,
        "skin_tone": true,
        "skin_disease": true
    },
    "uptime_seconds": 1234.5,
    "load_error": null
}

```

3. GET /api/models - Model Information

Returns detailed information about all 3 models including class labels, thresholds, and configuration.

Response (200):

```

{
  "status": "success",
  "models": {
    "skin_type": {
      "classes": ["combination", "dry", "normal", "oily"],
      "num_classes": 4,
      "checkpoint": "best_model.pth",
      "confidence_threshold": 0.5
    },
    "skin_tone": {
      "classes": ["dark", "fair", "medium"],
      "num_classes": 3,
      "checkpoint": "best_model.pt",
      "confidence_threshold": 0.5
    },
    "skin_disease": {
      "classes": ["common_acne", "cystic_acne", "eczema", "psoriasis", "rosacea", "tinea", "vitiligo"],
      "num_classes": 7,
      "checkpoint": "best_model.pth",
      "confidence_threshold": 0.4
    }
  },
  "device": "cpu",
  "image_size": 224,
  "max_upload_mb": 10,
  "allowed_formats": [".bmp", ".jpg", ".jpeg", ".png", ".webp"]
}

```

4. GET / - Frontend Upload Page

Serves the HTML frontend where users can drag-and-drop or select an image for analysis.

Frontend Interface

The frontend is a single-page HTML application at `templates/index.html` with:

- **Dark glassmorphism theme** with gradient accents
 - **Drag & drop upload zone** with animated border effects
 - **File picker button** (accepts image files only)
 - **Image preview** showing filename and size before analysis
 - **Animated loading spinner** during model inference
 - **Result cards** for each prediction with:
 - Predicted class name
 - Confidence percentage with color coding (green > 70%, yellow 50-70%, red < 50%)
 - Confidence bar visualization
 - Full probability distribution for all classes
 - **Skin profile summary** showing all 3 predictions at a glance
 - **Disease alert banner** (red if disease detected, green if clear)
 - **Warning badges** for low-confidence predictions
 - **Latency breakdown** showing timing of each step
 - **Responsive design** that works on mobile devices
-

Edge Cases Handled

Image Validation (`pipeline_engine.py`)

Edge Case	Error Code	HTTP Status	What Happens
No file uploaded	NO_FILE	400	Returns error asking to send file with key 'image'
Empty file (0 bytes)	EMPTY_FILE	400	Returns error immediately
File too large (> 10 MB)	FILE_TOO_LARGE	400	Returns error with file size in message
Unsupported format (.gif, .pdf, etc.)	UNSUPPORTED_FORMAT	400	Lists allowed formats in error message

Edge Case	Error Code	HTTP Status	What Happens
Corrupt/unreadable image	CORRUPT_IMAGE	400	PIL tried to open/decode and failed
Image too small (< 64x64)	TOO_SMALL	400	Returns dimensions in error message
File not found (CLI mode)	NOT_FOUND	400	Path doesn't exist

Model Edge Cases (pipeline_engine.py)

Edge Case	What Happens
Model checkpoint not found	ModelLoadError raised with path in message
Unknown checkpoint format	ModelLoadError with first 10 keys listed
Class count mismatch (checkpoint vs config)	Warning logged, uses checkpoint's value
Model output size != label count	Labels truncated to match output size
One model fails, others succeed	Returns partial_error (HTTP 207) with partial results
All models fail to load	Returns HTTP 503 Service Unavailable
Inference timeout (> 30 seconds)	Thread raises TimeoutError

Server Edge Cases (api/views.py)

Edge Case	HTTP Status	What Happens
Models not loaded yet	503	Returns SERVICE_UNAVAILABLE with detail
Unhandled Python exception	500	Caught by try/except, returns INTERNAL_ERROR
Wrong HTTP method (GET on /api/analyze)	405	Django's require_http_methods decorator
Cross-origin request	200	CORS middleware adds proper headers
OPTIONS preflight request	200	CORS middleware handles it

Confidence Thresholds

Model	Threshold	Behavior
Skin Type	50%	If below, adds warning to response
Skin Tone	50%	If below, adds warning to response
Skin Disease	40%	If below, adds warning AND <code>disease_detected = false</code>

The disease threshold is lower (40%) because medical predictions should be more cautious. The `disease_detected` flag is only `true` when BOTH: the prediction is NOT “none” AND the confidence is $\geq 40\%$.

How to Run

1. Install Dependencies

```
pip install torch torchvision timm pillow django
```

Or using the requirements file:

```
pip install -r requirements.txt
```

2. Verify Models are in Place

Make sure these files exist:

```
models/skin_type/best_model.pth
models/skin_tone/best_model.pt      <-- .pt NOT .pth
models/skin_disease/best_model.pth
```

3. Start the Django Server

```
python manage.py runserver 8000
```

You should see:

Watching for file changes with StatReloader

4. Open in Browser

Go to: **`http://localhost:8000`**

Upload an image using the drag-and-drop zone or file picker and click “Analyze Skin”.

5. Or Test via API

Health check

```
curl http://localhost:8000/api/health
```

Analyze an image

```
curl -X POST -F "image=@test_images/000045.jpg" http://localhost:8000/api/analyze
```

Model info

```
curl http://localhost:8000/api/models
```

6. Or Run the Test Script

```
python test_api.py
```

This tests all endpoints and analyzes all images in `test_images/`.

Testing

`test_api.py`

This script automatically tests all 3 API endpoints:

1. GET `/api/health` - Verifies models are loaded
2. GET `/api/models` - Verifies model configuration
3. POST `/api/analyze` - Analyzes every image in `test_images/` folder

Test Results

All 5 test images analyzed successfully:

Image	Skin Type	Skin Tone	Disease	Inference Time
000006.jpg	DRY (54%)	DARK (73%)	TINEA (94%)	233 ms
000014.jpg	DRY (57%)	FAIR (88%)	ROSACEA (61%)	244 ms
000045.jpg	DRY (54%)	MEDIUM (94%)	ECZEMA (86%)	332 ms
001021.jpg	NORMAL (46%)	MEDIUM (67%)	ECZEMA (98%)	275 ms
test_face_1.jpg	COMBINATION (39%)	FAIR (81%)	ACNE (81%)	266 ms

CLI Testing (Without Django)

You can also test the pipeline engine directly without starting the server:

```
python pipeline_engine.py test_images/000045.jpg
```

Configuration

All configuration is in the `Config` class inside `pipeline_engine.py`:

Setting	Default Value	Description
<code>IMAGE_SIZE</code>	224	Input image size for models
<code>CONFIDENCE_THRESH</code>	0.50 (50%)	General confidence threshold
<code>DISEASE_THRESH</code>	0.40 (40%)	Disease model confidence threshold
<code>DEVICE</code>	Auto-detected	Uses <code>cuda</code> if GPU available, else <code>cpu</code>
<code>MAX_WORKERS</code>	3	Number of parallel inference threads
<code>MAX_IMAGE_BYTES</code>	10 MB	Maximum upload file size
<code>ALLOWED_EXTENSIONS</code>	jpg, jpeg, .png, .webp, .bmp	Accepted file formats
<code>MIN_DIMENSION</code>	64	Minimum image width/height in pixels

Django settings are in `truetone/settings.py`:

Setting	Value	Description
<code>DEBUG</code>	True	Development mode
<code>ALLOWED_HOSTS</code>	['*']	Accepts all hosts (change in production)
<code>DATABASES</code>	{}	No database needed
<code>DATA_UPLOAD_MAX_MEMORY_SIZE</code>	10 MB	Django upload size limit
<code>TIME_ZONE</code>	Asia/Karachi	Server timezone

File-by-File Breakdown

`pipeline_engine.py` (Core Engine - 598 lines)

This is the brain of the entire system. It is framework-agnostic (works without Django too).

Sections: 1. **Config class** (lines 50-76) - All settings in one place 2. **Image transforms** (lines 79-84) - Resize, crop, normalize pipeline 3. **Exceptions**

(lines 90-97) - Custom error types with error codes 4. **Auto-detect functions** (lines 103-147) - Read checkpoint files to determine architecture 5. **Model builders** (lines 153-211) - Build the exact neural network from checkpoint info 6. **Model loader** (lines 217-265) - Load checkpoint, detect format, build model, load weights 7. **Image validation** (lines 271-320) - Validate extension, size, format, dimensions 8. **Inference** (lines 326-344) - Thread-safe forward pass with softmax and confidence check 9. **Aggregation** (lines 350-397) - Combine 3 model results into final response 10. **SkinAnalysisPipeline class** (lines 403-561) - Main orchestrator with parallel loading/inference 11. **Singleton** (lines 567-578) - Thread-safe single instance pattern 12. **CLI test** (lines 584-597) - Run from command line without Django

api/views.py (Django Views - 177 lines)

Four view functions that map HTTP requests to pipeline calls:

- `index()` - Renders the frontend HTML template
- `analyze_image()` - POST handler: extracts file, calls pipeline, returns JSON
- `health_check()` - GET handler: returns model status
- `model_info()` - GET handler: returns model configuration

api/middleware.py (CORS Middleware - 24 lines)

Adds `Access-Control-Allow-Origin: *` headers to all responses so the frontend can make API calls from any domain. Handles OPTIONS preflight requests.

api/urls.py (URL Routing - 12 lines)

Maps URL paths to view functions: - `/` -> `index` - `/api/analyze` -> `analyze_image` - `/api/health` -> `health_check` - `/api/models` -> `model_info`

truetone/settings.py (Django Config - 71 lines)

Minimal Django configuration: - No database (pure inference server) - CORS middleware enabled - 10 MB upload limit - Templates directory configured - Logging setup for truetone and api loggers

templates/index.html (Frontend - 280 lines)

Self-contained HTML/CSS/JS page with: - All styles inline (no external CSS files needed) - Google Inter font for typography - JavaScript for file handling, API calls, and result rendering - No framework dependencies (vanilla JS)

inspect_checkpoint.py (Utility - 60 lines)

Diagnostic tool that reads each model checkpoint and prints the classifier head structure. Useful for debugging architecture mismatches.

test_api.py (Test Script - 74 lines)

Automated test script that hits all API endpoints and prints formatted results for each test image.

API Response Format

Success Response Fields

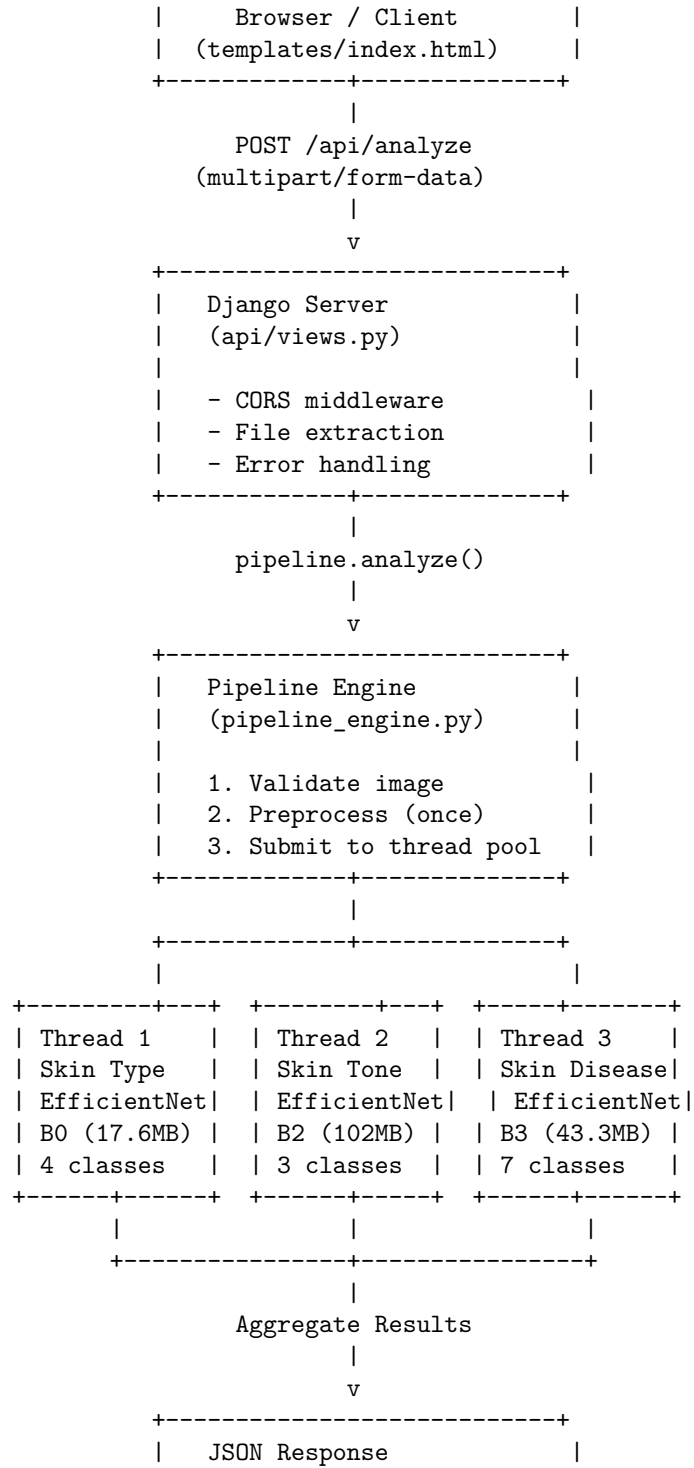
Field	Type	Description
status	string	Always “success”
request_id	string	Unique 8-char ID for this request
image_info.filename	string	Uploaded filename
image_info.size_bytes	int	File size in bytes
image_info.width	int	Image width in pixels
image_info.height	int	Image height in pixels
predictions.skin_type	label	Predicted skin type
predictions.skin_type_confidence	float	Confidence (0.0 to 1.0)
predictions.skin_type_distribution	dict	All class probabilities
predictions.skin_disease_disease_detected	bool	Whether disease found with sufficient confidence
skin_profile	object	Quick summary of all 3 predictions
warnings	array	List of warning messages (if any)
latency_ms	object	Timing breakdown in milliseconds

Error Response Fields

Field	Type	Description
status	string	“error” or “partial_error”
error_code	string	Machine-readable error type
message	string	Human-readable error description
request_id	string	Request tracking ID

Architecture Diagram

+-----+



```

| - predictions          |
| - confidence scores    |
| - warnings             |
| - latency breakdown     |
+-----+

```

Dependencies

Package	Version	Purpose
<code>torch</code>	<code>>= 2.0.0</code>	PyTorch deep learning framework
<code>torchvision</code>	<code>>= 0.15.0</code>	Pre-built model architectures (EfficientNet)
<code>timm</code>	<code>>= 0.9.0</code>	Additional model architectures (for Skin Tone model)
<code>Pillow</code>	<code>>= 9.0.0</code>	Image loading, validation, and preprocessing
<code>django</code>	<code>>= 4.2.0</code>	Web framework for REST API and frontend serving

Install all at once:

```
pip install torch torchvision timm pillow django
```

Notes

- The `models/` folder contains ML checkpoints, NOT Django ORM models. There is no database in this project.
- The first API request after server start takes 5-15 seconds (model loading). All subsequent requests are fast (~200-330ms).
- GPU is used automatically if CUDA is available. Otherwise falls back to CPU.
- The pipeline engine (`pipeline_engine.py`) can also be used standalone without Django by calling `pipeline.analyze_file("image.jpg")`.